Maximum Edge-based Quasi-Clique: Novel Iterative Frameworks

Hongbo Xia
Harbin Institute of Technology, Shenzhen
Shenzhen, China
24s151167@stu.hit.edu.cn

Shengxin Liu*
Harbin Institute of Technology, Shenzhen
Shenzhen, China
sxliu@hit.edu.cn

Zhaoquan Gu*
Harbin Institute of Technology, Shenzhen
Pengcheng Laboratory
Shenzhen, China
guzhaoquan@hit.edu.cn

Abstract

Extracting cohesive subgraphs from complex networks is a fundamental task in graph analytics and is essential for understanding biological, social, and web graphs. The edge-based γ-quasi-clique model offers a flexible alternative by identifying subgraphs whose edge densities exceed a specified threshold γ. However, finding the exact maximum edge-based quasi-clique is computationally challenging, as the problem is NP-hard and lacks the hereditary property. These characteristics limit the effectiveness of conventional pruning methods and the development of efficient reduction rules. As a result, existing algorithms, such as `QClique` and `FPCE`, struggle to scale to large graphs. In this paper, we revisit the problem and propose a novel iterative framework that reformulates the problem as a sequence of hereditary subproblems, enabling more effective pruning and reduction strategies and improving the worst-case time complexity. Furthermore, we redesign the iterative process and introduce a novel heuristic to further improve practical efficiency. Extensive experiments on 253 large-scale real-world graphs demonstrate that our proposed algorithm `EQC-Pro` outperforms existing methods by up to four orders of magnitude.

CCS Concepts

• **Theory of computation → Graph algorithms analysis**.

Keywords

Cohesive subgraph mining, edge-based quasi-clique

ACM Reference Format:
Hongbo Xia, Shengxin Liu, and Zhaoquan Gu. 2026. Maximum Edge-based Quasi-Clique: Novel Iterative Frameworks. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Many types of real-world information can be naturally represented as graphs, including biological protein networks, social networks, and web graphs. A fundamental problem in this context is the extraction of *cohesive subgraphs*. However, due to noise and incomplete edge information, the strict definition of a clique (i.e., a pairwise fully connected subgraph) is often too restrictive for practical scenarios. One intuitive relaxation is the *edge-based γ-quasi-clique* [2, 3], which introduces a parameter (between 0 and 1) to control the degree of relaxation. Specifically, a subgraph is considered an *edge-based γ-quasi-clique* if the ratio of its number of edges to that of a complete graph of the same size is at least γ. When $\gamma = 1$, this definition reduces to the classical clique model.

Recent studies [35] have provided strong empirical evidence that the edge-based γ-quasi-clique model offers practical benefits in biological network analysis. For instance, compared to the degree-based γ-quasi-clique model, which requires each vertex in the subgraph to be connected to at least a γ-fraction of the other vertices, the edge-based model achieves better predictive performance in identifying protein complexes, as measured by sensitivity, precision, and geometric accuracy [31]. Moreover, compared to heuristic approaches [31], clusters obtained from exact solutions exhibit higher biological relevance, as demonstrated by Gene Ontology enrichment[1] analysis, which reveals enrichment in more unique and functionally meaningful cellular components. These results highlight the practical advantages of exact dense subgraph mining for uncovering biologically significant structures. Beyond biological networks, the edge-based quasi-clique model has also proven effective in a variety of other contexts, including the analysis of social networks [5, 17] and the detection of anomalies [39, 45].

In this paper, we revisit the problem of *finding the exact maximum edge-based quasi-clique* [29, 36, 42]. Specifically, given a graph and a density threshold γ, our goal is to identify the largest edge-based γ-quasi-clique. This problem is challenging as it has been proven to be NP-hard [34], which creates substantial computational difficulties for exact solutions. Moreover, the edge-based γ-quasi-clique does not have the hereditary property; that is, its subgraphs are not necessarily edge-based γ-quasi-cliques. Consequently, many techniques that have been developed for finding cohesive subgraphs with the hereditary property, such as k-plexes [11, 12, 20] and k-defective cliques [8, 9, 16], cannot be directly applied to this problem. This limitation reduces the effectiveness of traditional pruning strategies and further increases the complexity of the problem.

State-of-the-Art Algorithms. Existing exact algorithms for the maximum edge-based quasi-clique problem fall into two main categories: mixed integer programming (MIP) and branch-and-bound (BNB) techniques. While MIP-based methods [29, 34, 42] can solve instances with up to 10^4 vertices, they often struggle to scale to

*Shengxin Liu and Zhaoquan Gu are the corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

[1] https://geneontology.org/

larger graphs in practice. Consequently, recent research has focused on developing efficient BNB algorithms.

Among BNB methods, QClique [36] stands out as the most efficient exact solver to date. It builds on the branching strategy of PCE [41], which exploits the quasi-hereditary structure of edge-based quasi-cliques [34]. QClique introduces a new upper bound to improve pruning, achieving performance competitive with earlier BNB and MIP-based approaches. In parallel, FPCE [35] represents the state-of-the-art algorithm for maximal enumeration and incorporates several refined pruning rules. Although FPCE is designed for enumerating maximal edge-based quasi-cliques, it can be adapted to the exact maximum problem by dynamically tracking the current best solution during search.

Despite these advances, both QClique and FPCE share certain limitations. Specifically, they inherit the same branching mechanism as PCE, making their pruning strategies tightly coupled to this structure. This coupling hinders the design of effective reduction rules for further efficiency improvements; to the best of our knowledge, no such reduction rules have been proposed in the literature. As a result, these solvers remain constrained on large graphs, where graph reduction is often decisive for performance.

Our New Methods. We reformulate the problem as a sequence of hereditary subproblems, enabling pruning and reduction techniques that explicitly leverage hereditary properties. Under this reformulation, we formally prove the correctness of the proposed iterative framework for the edge-based quasi-clique and establish its worst-case time complexity of $O^*(\beta_\kappa^n)$, where O^* suppresses polynomial factors, n is the number of vertices, $\beta_\kappa < 2$ is the largest real root of $x^{\kappa+2} - 2x^{\kappa+1} + 1 = 0$, and κ relates to the size of the optimal solution. This improves upon existing exact algorithms such as QClique and FPCE, both of which have $O^*(2^n)$ complexity. To the best of our knowledge, EQC-Pro is the first exact algorithm for MaxEQC with a non-trivial complexity.

Furthermore, by exploiting the quasi-hereditary property of the edge-based quasi-clique, we develop a bottom-up doubling iterative framework that reduces the number of iterations required to solve hereditary subproblems from $O(n)$ to $O(\log s^*)$ in theory, where s^* denotes the size of the optimal solution and is typically much smaller than n in practice. Finally, we design a simple yet effective heuristic grounded in the degeneracy sequence, a standard tool in cohesive subgraph mining. To mitigate the rigidity induced by strict adherence to a fixed degeneracy sequence, we introduce a dynamically refined heuristic strategy that iteratively updates the candidate solution via vertex additions and deletions. This strategy diversifies selection process and can yield better heuristic solutions.

Contributions. Our contributions are summarized as follows.

- We adapt the top-down iterative framework to the edge-based quasi-clique setting by reducing the problem to a sequence of hereditary subproblems, thereby achieving improved complexity of $O^*(\beta_\kappa^n)$. (Section 3)
- By leveraging the quasi-hereditary property of edge-based quasi-clique, we propose a bottom-up doubling iterative framework that reduces the number of iterations required to solve hereditary subproblems from $O(n)$ to $O(\log s^*)$. (Section 4)
- We design a novel heuristic strategy, further improving practical performance. (Section 5)

We conduct extensive experiments on 253 graphs and show that our proposed EQC-Pro achieves up to four orders of magnitude speedup over state-of-the-art baselines QClique and FPCE (Section 7). Our source code can be found in [1].

2 Preliminaries

Let $G = (V, E)$ be an undirected simple graph with $|V| = n$ vertices and $|E| = m$ edges. We denote $G[S]$ of G as the subgraph induced by the set of vertices $S \subseteq V$. For any $S \subseteq V$ and $u \in V$, let $d_G^S(u)$ denote the number of neighbors of u in S, i.e., $d_G^S(u) = |\{v \in S \mid v \neq u \text{ and } (u, v) \in E\}|$. Moreover, we use $N_G[u]$ to denote the closed neighborhood of vertex u in G, i.e., $N_G[u] = \{v \in V \mid v = u \text{ or } (u, v) \in E\}$, and $N_G[S]$ to denote the union of $N_G[u]$ for all vertices $u \in S$, i.e., $N_G[S] = \bigcup_{u \in S} N_G[u]$. For an induced graph g of G, the vertex set and the edge set of g are denoted by $V(g)$ and $E(g)$, respectively. For a subgraph g, we define *the number of missing edges* as the number of edges absent in g compared to a complete graph of the same size, i.e., $\binom{|V(g)|}{2} - |E(g)|$. In this paper, we focus on the edge-based γ-quasi-clique (γ-EQC) [2, 3].

Definition 1. *Given a real number $0 < \gamma < 1$, a graph g is said to be an* edge-based γ-quasi-clique *(γ-EQC) if $|E(g)| \geq \gamma \times \binom{|V(g)|}{2}$.*

In other words, an edge-based γ-quasi-clique is an induced subgraph in which the number of edges is at least a γ faction of the number of edges in the corresponding complete graph, i.e., $\binom{|V(g)|}{2}$. We are ready to present our problem in this paper.

Problem 1 (Maximum edge-based γ-quasi-clique). *Given a graph $G = (V, E)$ and a real number $\gamma \in (0, 1)$, the Maximum Edge-based γ-Quasi-Clique Problem (*MaxEQC*) aims to find an edge-based γ-quasi-clique in G with the largest number of vertices.*

Let g^* and $s^* = |V(g^*)|$ be the maximum γ-EQC and its size. Finding the maximum γ-EQC is known to be NP-hard [34]. Moreover, γ-EQCs lack the *hereditary property* [34]: A subgraph of a γ-EQC is not necessarily a γ-EQC. This limitation complicates efficient algorithm design, as even checking maximality by inclusion becomes a non-trivial task for γ-EQC. Nevertheless, γ-EQCs satisfy a weaker structural guarantee, known as the *quasi-hereditary property*.

Proposition 1 (Quasi-hereditary property [34]). *Let $G = (V, E)$ be a graph and $S \subseteq V$ such that $G[S]$ is a γ-EQC. If there exists a vertex $v \in S$ with degree less than the average degree in $G[S]$, i.e., $d_G^S(v) \leq \frac{2|E(G[S])|}{|S|}$, then $G[S']$, where $S' = S \setminus \{v\}$, is a γ-EQC.*

Proposition 1 implies that for a given γ-EQC g, removing a vertex satisfying a specific degree constraint guarantees that the remaining graph remains a γ-EQC, whereas removing an arbitrary vertex does not, due to the non-hereditary property of γ-EQCs.

3 A Top-down Iterative Baseline Framework

A closely related problem to MaxEQC is the *maximum degree-based quasi-clique problem (*MaxQC*)*, which has been effectively addressed by the state-of-the-art iterative algorithm IterQC [43]. We note that neither of these two problems satisfies the hereditary property. Thus, inspired by the key iterative idea behind IterQC, our first framework seeks to transform the MaxEQC problem into a sequence of subproblems that do possess the hereditary property, following

a top-down approach. We introduce the cohesive subgraph of k-defective clique, which serves as a key component in our approach.

Definition 2 ([44]). *Given an integer $k \geq 1$, a graph g is said to be a k-*defective clique *if $|E(g)| \geq \binom{|V(g)|}{2} - k$.*

In other words, a k-defective clique g contains at most k missing edges. Note that the k-defective clique enjoys the hereditary property, which can be leveraged to prune the search space. Building on the concept of k-defective clique, extensive studies [8, 9, 14, 15, 19, 28] have focused on the following problem.

Problem 2 (Maximum k-defective clique). *Given a graph $G = (V, E)$ and a positive integer k, the Maximum k-Defective-Clique Problem (*kDC*) aims to find the largest k-defective-clique in G.*

Based on kDC, we can define the following two functions.

- $\texttt{get-k}(n) := \lfloor (1-\gamma) \cdot \binom{n}{2} \rfloor$, which takes a number n of vertices as input and returns an appropriate value of k;
- $\texttt{solve-defect}(k) :=$ the size of the largest k-defective clique in G, which takes a value k as input.

Inspired by IterQC [43], our top-down iterative approach EQC-TD starts from an upper bound $s_0 > s^*$ and alternately applies $k_i = \texttt{get-k}(s_{i-1})$ and $s_i = \texttt{solve-defect}(k_i)$ until convergence, i.e., until $s_i = s_{i-1}$. It can be shown that the sequence s_i is monotonically decreasing and converges to the optimal value s^*. Thus, EQC-TD correctly reduces the MaxEQC problem, which lacks the hereditary property, to a sequence of kDC instances, which are hereditary. Due to space constraints, the pseudocode and correctness proof of EQC-TD are provided in Appendix A.

Limitations. The framework still suffers from the efficiency issues. **First**, in the worst case, EQC-TD requires n calls to solve-defect, leading to excessive number of iterations. **Second**, although kDC is hereditary, its computational cost becomes prohibitive as k increases. Specifically, for a fixed density γ, the get-k function indicates that k grows quadratically with the number of vertices.

4 An Advanced Bottom-up Iterative Framework

To address the limitations of the top-down iterative framework, we leverage a structural property, i.e., the quasi-hereditary property in Proposition 1, unique to the definition of γ-EQC, which is absent under the degree-based formulation. Based on this property, we develop a specially designed bottom-up iterative framework tailored for MaxEQC. We first introduce an important property of the iterative framework for γ-EQC, which is derived from Proposition 1.

Proposition 2. *Recall that s^* denote the size of the maximum edge-based γ-quasi-clique. Then, it holds that*
***(1)** for any $s \leq s^*$, $s \leq \texttt{solve-defect}(\texttt{get-k}(s))$, and*
***(2)** for any $s > s^*$, $s > \texttt{solve-defect}(\texttt{get-k}(s))$.*

Proof. We recall that the function $\texttt{solve-defect}(k)$ returns the *largest* integer s' such that there exists a k-defective clique of size s'. We then consider two cases according to the relationship between s and s^*. In both cases, we let $k = \texttt{get-k}(s) = \lfloor (1-\gamma)\binom{s}{2} \rfloor$.

Case 1: $s \leq s^*$. We show that it is always possible to construct a γ-EQC of size s from the maximum γ-EQC g^* of size s^*, such that $s \leq \texttt{solve-defect}(\texttt{get-k}(s))$ holds. Specifically, we begin with g^* and iteratively remove a vertex of minimum degree from the subgraph until only s vertices remain. Since a vertex of minimum degree always has less than or equal to the average degree, the quasi-hereditary property (Proposition 1) ensures that the induced subgraph g on the remaining s vertices remains a γ-EQC. Thus, the number of edges in g is at least $\gamma \times \binom{s}{2}$, which implies that g is a feasible k-defective clique of size s since the number of missing edges is $\lfloor \binom{s}{2} - \gamma \times \binom{s}{2} \rfloor = k$. Consequently, we have $\texttt{solve-defect}(k) \geq s$.

Case 2: $s > s^*$. We prove by contradiction. Suppose, to the contrary, that $s \leq \texttt{solve-defect}(\texttt{get-k}(s))$. Let $s' = \texttt{solve-defect}(\texttt{get-k}(s))$, and let g' be the corresponding k-defective clique returned. By definition of a k-defective clique (Definition 2), the number of edges in g' satisfies $E(g') \geq \binom{s'}{2} - \texttt{get-k}(s)$. Since $\texttt{get-k}(s) \leq \texttt{get-k}(s') = \left\lfloor (1-\gamma)\binom{s'}{2} \right\rfloor$, we have $E(g') \geq \binom{s'}{2} - \left\lfloor (1-\gamma)\binom{s'}{2} \right\rfloor \geq \gamma\binom{s'}{2}$. Thus, g' corresponds to a γ-EQC of size s', and we know $s' \geq s > s^*$, which contradicts the maximality of s^*. □

Bottom-up Framework Overview. Proposition 2 implies that we can identify s^* by carefully analyzing the relationship between s and $\texttt{solve-defect}(\texttt{get-k}(s))$. In particular, based on Proposition 2, we can devise a more efficient iterative framework by performing a binary search on the candidate size s. Consequently, the number of iterations to determine s^* can be reduced from $O(n)$ to $O(\log n)$, thereby improving efficiency. Nevertheless, it is important to note that the efficiency of $\texttt{solve-defect}(k)$ is highly dependent on the value of k. Specifically, the problem becomes easier to solve as k decreases. This observation motivates us to explore alternative search strategies that can further reduce the computational cost, especially in the early stages of the iterative framework.

To this end, we make use of a *bottom-up doubling strategy* to progressively approach the optimal value s^*. Specifically, let lb and ub denote the current lower and upper bounds of s^*. The standard binary search approach would iteratively select the midpoint, $s = \lfloor (lb+ub)/2 \rfloor$, as the candidate value of s to evaluate at each iteration, updating the bounds based on the result. While this method is efficient in terms of the number of iterations, it may not always be efficient in practice, particularly when we need to invoke solve-defect with a large value of k (e.g., when ub is large).

In contrast, our bottom-up doubling strategy begins by probing the values of s in an exponentially increasing sequence $\{lb+1, lb+2, lb+4, \ldots\}$, doubling the step size at each iteration until the value exceeds s^*. After a candidate interval that must contain s^* is identified, we then switch to a more localized search, i.e., we perform a binary search within this narrowed interval. This approach keeps the values of k encountered in the early rounds as small as possible, which can potentially improve the overall efficiency of the bottom-up doubling strategy.

Algorithm. Our bottom-up doubling iterative framework EQC-BU is detailed in Algorithm 1. EQC-BU searches for the optimal value s^* by dynamically adjusting the lower bound lb and upper bound ub of s^*. In Line 1, EQC-BU initializes lb to 1 and ub to $|V|$, as it holds that $1 \leq s^* \leq |V|$. The core idea of EQC-BU is to efficiently and progressively identify the search interval containing s^* via the check subroutine (described in Lines 14-17). This subroutine evaluates whether a candidate value of s does not exceed the optimal solution size s^*, as guaranteed by Proposition 2. The evaluation process has two phases: a doubling phase and a halving phase.

Algorithm 1: A Bottom-up Doubling Framework: EQC-BU

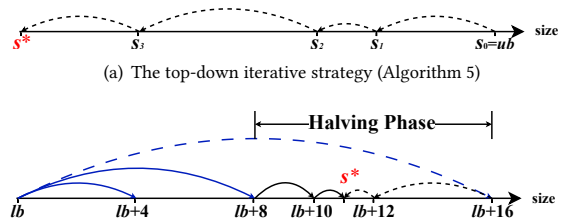
Input: A graph $G = (V, E)$ and a real value $0 < \gamma < 1$
Output: The size of the maximum γ-EQC in G

1 $lb \leftarrow 1$; $ub \leftarrow |V|$;
// Doubling Phase
2 $gap \leftarrow 1$;
3 **while** $lb + gap \leq ub$ **and** check$(lb + gap)$ **do**
4 $\quad lb_{\text{new}} \leftarrow lb + gap$;
5 $\quad gap \leftarrow 2 \times gap$;
// Halving Phase
6 $ub_{\text{new}} \leftarrow \min(lb + gap - 1, ub)$;
7 **while** $lb_{new} \neq ub_{new}$ **do**
8 $\quad mid \leftarrow \lceil (lb_{\text{new}} + ub_{\text{new}})/2 \rceil$;
9 $\quad$ **if** check(mid) **then**
10 $\quad\quad lb_{\text{new}} \leftarrow mid$;
11 $\quad$ **else**
12 $\quad\quad ub_{\text{new}} \leftarrow mid - 1$;
13 **return** lb_{new};
14 **Function check(s):**
15 $\quad k \leftarrow$ get-k(s);
16 $\quad s' \leftarrow$ solve-defect(k);
17 $\quad$ **return** $s' \geq s$;

(1) Doubling Phase (Lines 2-5). The doubling phase initializes the step size gap to 1 (Line 2) and iteratively calls check$(lb + gap)$ for increasing values of gap if $lb + gap$ does not exceed the current upper bound ub (Line 3). If check returns true, then by Proposition 2, $lb + gap$ is feasible, which implies $lb + gap \leq s^*$. This value is then used to update the lower bound lb_{new} (Line 4), which will be the starting point for the halving phase. Then, in Line 5, the algorithm doubles gap to expand the interval upward, aiming to quickly approach or exceed s^*. This process repeats until either the feasibility check fails or ub is reached in Line 3. Note that if the current candidate value of $lb+gap$ is infeasible, we know $s^* < lb+gap$, which will be used to update ub_{new} in the halving phase.

(2) Halving Phase (Lines 6-12). After the doubling phase terminates, the algorithm proceeds to a binary search (halving) phase to precisely locate the optimal value s^*. In this phase, the refined interval $[lb_{\text{new}}, ub_{\text{new}}]$ is set according to the final state after doubling (Line 6). The halving phase then performs a standard binary search within this narrowed interval. In each iteration, it tests the midpoint $mid = \lceil (lb_{\text{new}} + ub_{\text{new}})/2 \rceil$. If check$(mid)$ returns true, the lower bound lb_{new} is updated to mid; otherwise, the upper bound ub_{new} is set to $mid - 1$ (Lines 7-12). This binary refinement continues until $lb_{\text{new}} = ub_{\text{new}}$, at which point the optimal value s^* is identified. Finally, EQC-BU returns lb_{new} as s^* in Line 13.

Comparison of Top-Down and Bottom-Up Strategies. To facilitate a clearer comparison between the top-down and bottom-up iterative frameworks, Figure 1 presents both strategies. Dashed arrows indicate updates to the upper bound, while solid arrows represent improvements to the lower bound. Figure 1(a) illustrates the top-down iterative strategy, in which it attempts to tighten the upper bound on the solution size. Dashed arrows represent these



(a) The top-down iterative strategy (Algorithm 5)

(b) The bottom-up doubling iterative strategy (Algorithm 1)

Figure 1: Illustration of iterative frameworks.

successive updates, which continue until the bound converges to the optimal size s^*. However, since each step does not guarantee a fixed reduction, the rate of progress may vary considerably across iterations. For example, there may be a substantial decrease from s_2 to s_3, or only a negligible improvement from s_1 to s_2. The bottom-up doubling approach (Figure 1(b)) begins by rapidly identifying a solution interval through the doubling phase (shown in blue), and then performs the having phase (shown in black) within that interval to locate the optimal solution. Both approaches require $O(\log s^*)$ iterations in the worst case.

Correctness and Remarks. We remark that EQC-BU consists of a doubling phase, which quickly finds an upper bound of s^*, followed by a halving phase that efficiently narrows the candidate interval containing s^*. EQC-BU is specifically tailored for our iterative framework for the MaxEQC problem, where the computation performed by the invoked kDC solver becomes increasingly costly as k grows.

For the correctness of EQC-BU (Algorithm 1), we observe that, by Proposition 2, check(s) is monotonic in s: once it fails at some s, it fails for all larger values; conversely, once it succeeds, it succeeds for all smaller values. After the doubling phase (Lines 2-5), we observe that the optimal value s^* always lies in the interval $[lb_{\text{new}}, ub_{\text{new}}]$. This is because (1) lb_{new} is only increased when a feasible candidate is found in Lines 2-5, and (2) $ub_{\text{new}} = \min(lb + gap - 1, ub)$ is set in Line 6 when $lb + gap > ub$ or an infeasible candidate is detected in Line 3. We then enter the halving (i.e., binary search) phase on this narrowed interval. In each iteration of the binary search, the midpoint test either raises the lower bound or lowers the upper bound while preserving the invariant. The process terminates precisely when the two bounds coincide, at which point the common value must be s^*. Thus, EQC-BU is correct.

We then analyze the number of calls to check in EQC-BU (and will discuss the time complexity in Section 6). The doubling phase (Lines 2-5) performs at most $\lceil \log_2 s^* \rceil + 1$ calls to check, since the algorithm starts from a small candidate size (e.g., 1) and doubles the size until it exceeds the actual optimal size s^*. Once a valid upper bound is detected, where this upper bound is clearly at least s^* and at most $2s^*$, the doubling phase terminates. In the subsequent halving phase (Lines 6-12), the search interval lies strictly below $2s^*$ and is halved in each iteration. Thus, the number of iterations in this phase is also bounded by $\lceil \log_2 s^* \rceil$. Overall, the total number of calls to check is bounded by $O(\log s^*)$.

5 Our Novel Heuristic Strategies

Since our bottom-up framework EQC-BU (Section 4) initiates iterations from a lower bound, i.e., a value smaller than the optimal value s^*, obtaining a larger lower bound in polynomial time through heuristics can reduce the total number of required iterations and

Algorithm 2: Degen-Opt(G, γ)

Output: A large γ-EQC in G
1 $g \leftarrow$ Degen(G, γ);
2 **foreach** *vertex* $u \in V$ **do**
3 $\quad g' \leftarrow$ Degen$(G[N_G[u]], \gamma)$;
4 $\quad$ **if** $|V(g')| > |V(g)|$ **then** $g \leftarrow g'$;
5 **return** g;
6 **Function** Degen(G, γ):
7 $\quad$ Compute a degeneracy ordering in G;
8 $\quad H \leftarrow$ the longest suffix of the degeneracy ordering such that $G[H]$ is a γ-EQC;
9 $\quad$ **return** $G[H]$;

thus improve overall efficiency. In this section, we propose novel heuristic strategies to compute a large lower bound for MaxEQC.

5.1 Basic Heuristic Methods

We present two basic heuristic methods Degen and Degen-Opt in Algorithm 2, both adapted from the heuristics in KDC [8]. These methods utilize the concept of the *degeneracy ordering*, which is obtained by iteratively removing the vertex with the smallest degree from the graph [4]. The first basic heuristic algorithm Degen (Lines 6-9 of Algorithm 2) iteratively removes the vertex with the minimum degree until the remaining subgraph forms a γ-EQC. However, Degen has a structural limitation: once a vertex is selected into the solution, *all subsequent vertices in the degeneracy ordering are inevitably included as well.* This inflexible selection mechanism results in a rigid combination process, thereby restricting the ability of the algorithm to explore more promising candidate subgraphs.

Degen-Opt in Algorithm 2 partially relaxes this restriction. Specifically, for each vertex u in the graph, we consider the subgraph induced by its closed neighborhood, i.e., $N_G[u] = \{v \in V \mid v = u \text{ or } (u, v) \in E\}$, and execute Degen on this subgraph. In this way, each vertex has more opportunities to be included in the solution. Nevertheless, while Degen-Opt expands the combination space by constructing multiple sequences and thereby allows more flexibility in vertex selection, it still relies on fixed suffixes of degeneracy sequences. Moreover, since Degen-Opt is applied only to subgraphs induced by closed neighborhoods, the diameter of any resulting solution is at most 2, which is not a necessary condition for edge-based quasi-cliques. Thus, although Degen-Opt alleviates the rigidity of Degen to some extent, it does not resolve the limitation and additionally imposes an unnecessary restriction on solution diameter.

5.2 Our Dynamic Refined Heuristic Strategies

To further mitigate the inflexibility caused by the combination of Degen-Opt, we proposed our dynamic refined heuristic method EQC-Heu-Pro, which *dynamically refines the heuristic solution by iteratively removing and adding vertices.* This strategy is designed to overcome the constraints on candidate solution construction.

Algorithmic Overview. Our dynamic refined heuristic strategies take as input a feasible γ-EQC g and iteratively adjust its vertex set to potentially obtain a larger heuristic solution. The core idea is to dynamically add and remove vertices between the selected set

Algorithm 3: EQC-Heu(G, γ, g)

Input: A graph $G = (V, E)$, a real value $0 < \gamma < 1$, and a heuristic solution g
Output: An improved γ-EQC in G
1 $g' \leftarrow g; k \leftarrow$ get-k$(|V(g')| + 1)$;
2 **foreach** *vertex* $u \in V(g)$ **do**
3 $\quad S \leftarrow \{u\}; C \leftarrow V \setminus \{u\}$;
4 $\quad$ **while** $|E(G[S])| \geq \binom{|S|}{2} - k$ **do**
5 $\quad\quad$ **if** $|S| > |V(g')|$ **then**
6 $\quad\quad\quad g' \leftarrow G[S]; k \leftarrow$ get-k$(|V(g')| + 1)$;
7 $\quad\quad$ AddBestVertex(S, C); AddBestVertex(S, C);
8 $\quad\quad$ RemoveWorstVertex(S, C);
9 $\quad\quad$ UpdateScore(S, C);
10 **return** g';

S and the candidate set C, while maintaining the γ-EQC property throughout the process. Specifically, for each vertex in g, we use it as a starting point (i.e., a *seed*) and initialize the selected set with this seed. At each iteration, we update the selected set by *adding two vertices and removing one vertex*, guided by a score function that aims to maximize the quality of the solution. After every update, we verify that the subgraph induced by the selected set remains a γ-EQC. This iterative process proceeds until the number of missing edges in the subgraph induced by the selected set exceeds the allowable threshold. In particular, given the largest γ-EQC g' identified so far, we compute the maximum number of missing edges permitted for a γ-EQC of size $|V(g')| + 1$, since our goal is to find a larger one. After evaluating every vertex in g as a seed, we select the largest γ-EQC found during this process as our final solution.

We note that the score function is designed to maximize the size of the heuristic solution by prioritizing the addition of vertices that increase the density of the subgraph induced by the selected set and the removal of those that decrease it. We adopt the scoring function from [13], which is detailed in Appendix B for completeness.

EQC-Heu. Our heuristic strategy EQC-Heu is summarized in Algorithm 3. We first initialize the current largest γ-EQC g' and the maximum number k of missing edges permitted for a γ-EQC of size $|V(g')| + 1$ in Line 1 and consider each vertex u in $V(g)$ as a seed in Lines 2-10. For each u, we initialize the selected set $S \leftarrow \{u\}$ and the candidate set $C \leftarrow V \setminus S$ in Line 3. We then iteratively update S and C by adding two vertices from C to S (Line 7) and removing one vertex from S to C (Line 8), using AddBestVertex and RemoveWorstVertex, which select vertices according to their scores. Then the UpdateScore function computes the score for each vertex based on the current selected set S and the candidate set C. (The details of AddBestVertex, RemoveWorstVertex, and UpdateScore are shown in Appendix B.) Importantly, after every update, we ensure that the subgraph induced by S continues to satisfy the γ-EQC property (Line 4). Once a larger γ-EQC is found, we update g' accordingly in Lines 5-6. The process repeats until the number of missing edges in the current selected set exceeds the allowable threshold, which is recalculated based on the size of the largest γ-EQC found so far. Finally, after all vertices in $V(g)$ are considered as seeds, we return the largest γ-EQC found (Line 10).

Algorithm 4: EQC-Heu-Pro(G, γ, g)

Input: A graph $G = (V, E)$, a real value $0 < \gamma < 1$, and a heuristic solution g
Output: An improved γ-EQC in G

1 **while** *true* **do**
2 $\quad g' \leftarrow$ EQC-Heu$(G[N_G[V(g)]], \gamma, g)$;
3 $\quad$ **if** $|V(g')| = |V(g)|$ **then break**;
4 $\quad g \leftarrow g'$;
5 **return** g;

Improved EQC-Heu-Pro. Although EQC-Heu alleviates the issue of inflexible vertex combinations, applying it directly to the entire graph is inefficient. For large graphs, maintaining set-related information incurs substantial memory overhead, and the associated random access patterns degrade cache performance, thereby reducing overall efficiency. To address these challenges, we propose EQC-Heu-Pro, which applies the heuristic to multiple smaller subgraphs rather than the entire graph, as in Algorithm 4. Specifically, EQC-Heu-Pro first extracts the subgraph induced by the closed neighborhood $N_G[V(g)]$ of the current solution g (Line 2), and then applies EQC-Heu to this induced subgraph to obtain a refined solution g'. If no improvement in size is achieved (i.e., $|V(g')| = |V(g)|$), the process terminates (Line 3); otherwise, g is updated to g' and the procedure is repeated (Line 4). This iterative refinement ensures efficiency and addresses the problem of inflexible solution construction without compromising solution quality.

Time Complexity Analyses. We briefly analyze the time complexities of EQC-Heu and EQC-Heu-Pro. First, in Algorithm 2, computing degeneracy orderings on neighborhood subgraphs gives overall time $O(\Delta_G m)$, where Δ_G is the maximum degree. For EQC-Heu (Algorithm 3), the main cost lies in updating scores for neighbors of the current set S. Each update takes $O(\Delta_G s^*)$ time, and with at most $(s^*)^2$ iterations, the total time is $O(\Delta_G (s^*)^3)$. Finally, EQC-Heu-Pro (Algorithm 4) calls EQC-Heu up to s^* times, yielding $O(\Delta_G (s^*)^4)$.

6 Our Final Iterative Algorithm: EQC-Pro

Building on the insights from the bottom-up iterative framework (Section 4) and heuristic strategies (Section 5), we introduce our final algorithm EQC-Pro. We first outline two key observations that guide the design of EQC-Pro and improve its practical performance.

Observation 1. Additional information can be derived from the function solve-defect(k). In particular, when the check function in Algorithm 1 returns true, a solution to the kDC problem can be constructed directly. Moreover, since get-k$(\cdot)$ is a floor function, multiple values of s may map to the same k, allowing solutions of kDC for one s to be reused for other s with the same k.

Observation 2. Within a single iteration in Algorithm 1, the lower bound s_i obtained corresponds to the solution of the kDC problem for a particular value of k. However, since our ultimate objective is to determine the lower bound s^* for the MaxEQC problem, there remains potential for improvement by leveraging this relationship.

Based on these observations, we (1) cache the results for previously solved values of k to enable direct reuse in subsequent iterations, and (2) apply EQC-Heu-Pro to extend the solution obtained in each iteration, thereby potentially obtaining a larger lower bound for MaxEQC. Due to space limitations, the detailed pseudocode and implementation of EQC-Pro are provided in Appendix C.

Remarks and Complexity Analyses. We note that get-k can be directly implemented, while solve-defect can leverage existing kDC algorithms [8, 9, 15]. In our implementation, we adopt the state-of-the-art algorithm kDC-two [9] for solve-defect. For the time complexity of EQC-Pro, we note that EQC-Pro essentially implements the bottom-up doubling iterative framework EQC-BU described in Section 4. As discussed there, the number of iterations is $O(\log s^*)$. In each iteration, the algorithm invokes solve-defect, which dominates the computational cost, while the accompanying heuristic method EQC-Heu-Pro runs in polynomial time and is invoked only once per iteration. The complexity of kDC-two (used for solve-defect in our implementation) is $O^*(\beta_k^n)$ [9], where β_k is the largest real root of the equation $x^{k+2} - 2x^{k+1} + 1 = 0$, and O^* omits polynomial factors. Note that the complexity of solve-defect increases monotonically with k. By our analysis of EQC-BU, the largest k that EQC-Pro needs to handle is $\kappa =$ get-k$(2s^*)$. Thus, the complexity of EQC-Pro is $O^*(\beta_\kappa^n)$, which is asymptotically better than the complexities of both QClique and FPCE, each of which has complexity $O^*(2^n)$. For space complexity, EQC-Pro is $O(m+n)$ as both kDC-two and the heuristic components such as EQC-Heu-Pro maintain linear-sized structures.

7 Experiments

Algorithms. We compare EQC-Pro with state-of-the-art **exact** methods for the maximum **edge-based** quasi-clique problem (MaxEQC).

- QClique[2]: the state-of-the-art branch-and-bound algorithm [36].
- FPCE[3]: a baseline adapted from the algorithm for enumerating all maximal γ-EQCs above a given size threshold [35]. We modify it for MaxEQC by maintaining the largest solution found so far and updating the lower bound accordingly to improve pruning.

We also evaluate the proposed bottom-up framework (Section 4) and the proposed heuristic method EQC-Heu-Pro (Section 5).

- EQC-TD: Algorithm 5, the top-down iterative approach.
- EQC-NH: A variant of EQC-Pro designed to quantify the contribution of EQC-Heu-Pro. All components in EQC-Pro that involve the use of EQC-Heu-Pro are removed in this variant.

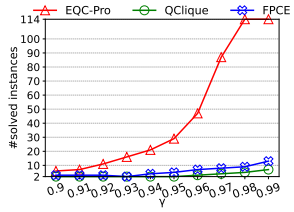
All algorithms are coded in C++ and compiled using g++ with -O3. Experiments are conducted on a machine with an Intel 2.60GHz CPU and 256GB RAM, running Ubuntu 20.04.6. We enforce a maximum runtime of 3 hours (10,800 seconds) per instance.

Datasets. We conduct experiments on the two collections from Network Repository, which are widely used in related studies. (1) **Real-World Collection**[4]: 139 real-world graphs; and (2) **Facebook Collection**[5]: 114 Facebook social networks.

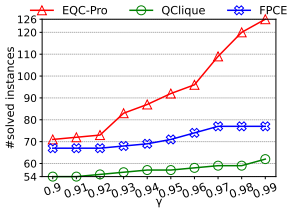
7.1 Comparison with Existing Algorithms

Number of Solved Instances. We note that an instance is considered *solved* only if an exact solution is found within the given time

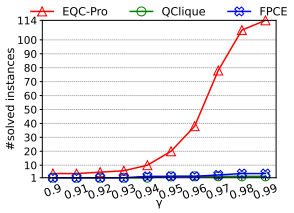
[2]We re-implemented the algorithm based on the description in [36]. Our implementation achieves better performance compared to the results reported in their paper.
[3]https://github.com/ahsanur-research/FPCE
[4]http://lcs.ios.ac.cn/~caisw/Resource/realworld%20graphs.tar.gz
[5]https://networkrepository.com/socfb.php



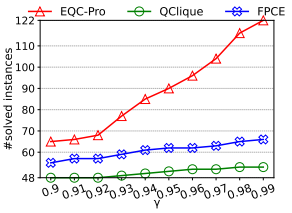
(a) Facebook (3-hour limit)



(b) Real-World (3-hour limit)

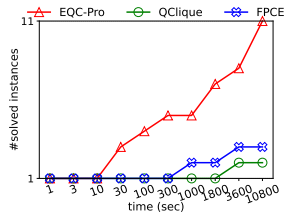


(c) Facebook (300-second limit)

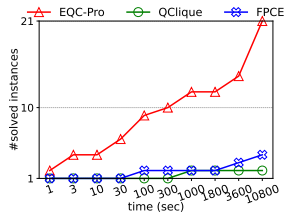


(d) Real-World (300-second limit)

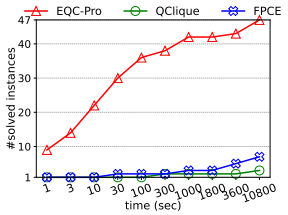
Figure 2: Number of solved instances with varying γ.



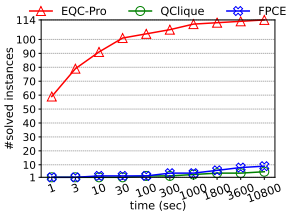
(a) $\gamma = 0.92$



(b) $\gamma = 0.94$



(c) $\gamma = 0.96$



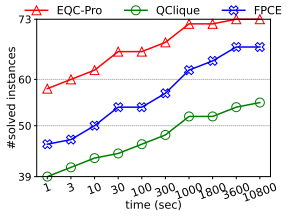
(d) $\gamma = 0.98$

Figure 3: Number of solved instances on Facebook.

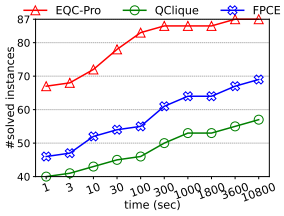
Table 1: Runtime (in seconds) on 20 large graphs with $\gamma = 0.95$.

Graphs	EQC-Pro	EQC-TD	EQC-NH	QClique	FPCE	n	m	s^*
inf-road-usa	7.16	7.52	**5.93**	–	358.36	23M	57M	4
inf-roadNet-CA	0.57	0.62	**0.49**	–	3.58	1.9M	5M	4
web-wikipedia2009	**8.03**	–	39.10	–	–	1.8M	9M	34
soc-pokec	**205.40**	–	460.59	–	–	1.6M	44M	35
soc-lastfm	**19.66**	–	79.49	–	–	1.1M	9M	20
soc-youtube-snap	**6.88**	–	20.00	–	–	1.1M	5.9M	22
rt-retweet-crawl	1.44	**1.13**	1.26	–	11.93	1.1M	4.5M	16
inf-roadNet-PA	**0.31**	0.46	0.39	–	1.37	1M	3M	4
sc-ldoor	**211.23**	–	212.22	–	–	909K	41M	22
soc-delicious	**0.67**	–	4.64	–	–	536K	2.7M	30
soc-youtube	**4.31**	–	12.79	–	–	495K	3M	22
sc-msdoor	**101.36**	–	101.92	–	–	404K	18M	22
soc-twitter-follows	0.24	**0.16**	0.25	3335.43	5.28	404K	1.4M	7
ca-MathSciNet	**0.18**	–	0.19	–	158.14	332K	1.6M	25
sc-pwtk	**55.5**	–	100.45	–	–	217K	11M	26
socfb-konect	**22.82**	23.38	28.99	–	1902.99	59M	185M	7
socfb-uci-uni	**28.15**	29.59	33.39	–	–	58M	184M	7
socfb-A-anon	**6857.26**	–	–	–	–	3M	47M	37
socfb-B-anon	**1952.97**	–	–	–	–	2.9M	41M	35
socfb-Maine59	**2421.99**	–	3247.84	–	–	9K	486K	41

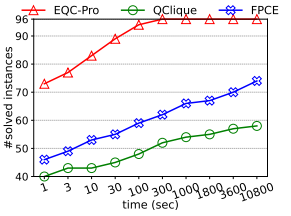
limit. Figure 2 presents the number of instances solved by EQC-Pro and two baseline methods, QClique and FPCE, under 3-hour and



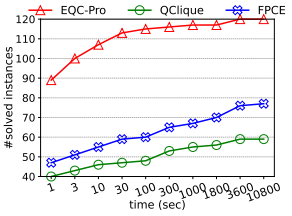
(a) $\gamma = 0.92$



(b) $\gamma = 0.94$



(c) $\gamma = 0.96$



(d) $\gamma = 0.98$

Figure 4: Number of solved instances on Real-World.

300-second time limits, respectively, with varying γ values from 0.9 to 0.99. We have the following observations. **First**, the number of solved instances sharply decreases as γ becomes smaller. This is because a lower value of γ corresponds to a looser constraint on γ-EQC, which significantly weakens the effectiveness of pruning and reduction rules in branch-and-bound methods. **Second**, EQC-Pro consistently outperforms the two baselines across all tested values of γ. For example, in Figure 2(a), when $\gamma = 0.98$ on the Facebook collection, EQC-Pro solves all 114 instances, while QClique and FPCE only solve 5 and 9 instances, respectively. For the Real-World collection, at $\gamma = 0.9$ under the 300-second time limit, QClique and FPCE solve 48 and 55 instances, respectively, whereas EQC-Pro solves 65 instances as shown in Figure 2(d). The significant speedup achieved by EQC-Pro results from transforming the problem into a sequence of hereditary subproblems, which substantially accelerates the search process. Compared to the Real-World collection, the Facebook graphs are denser, making the problem more challenging to solve under the same values of γ. The advantage of EQC-Pro becomes more obvious at higher γ, where the problem is moderately constrained. However, as γ decreases, the search space increases dramatically, resulting in greater computational difficulty and reducing the performance gap between EQC-Pro and the baselines.

Figures 3 and 4 illustrate the number of solved instances over time on the Facebook and Real-World collections, respectively, for $\gamma \in \{0.92, 0.94, 0.96, 0.98\}$. As shown, both baseline methods consistently exhibit poor performance on these datasets regardless of the value of γ. On the Facebook collection, when $\gamma = 0.98$, EQC-Pro successfully solves all 114 instances, whereas the best of the two baselines solves only 9. Moreover, across all tested values of γ, EQC-Pro solves more instances within 30 seconds than either baseline does within 3 hours. For example, at $\gamma = 0.96$, EQC-Pro solves 30 instances in only 30 seconds, compared to only 3 and 7 solved by QClique and FPCE, respectively, even with a 3-hour limit. On the Real-World collection, EQC-Pro also consistently outperforms the two baselines under all tested values of γ. Specifically, at $\gamma = 0.94$, EQC-Pro solves 77 instances within only 30 seconds, while the best baseline solves only 68 instances even with a 3-hour limit.

Evaluation on Large Graphs. Table 1 reports the performance of the baselines QClique and FPCE, as well as our EQC-Pro (and its variants), on large graphs with $\gamma = 0.95$. Specifically, we select 15 largest graphs from the Real-World collection and 5 largest graphs from the Facebook collection, excluding extremely difficult cases in which all three algorithms fail to solve. The instances are then sorted in descending order of graph size in Table 1. We use "–" to indicate a timeout (i.e., 3 hours) and highlight the best results in **bold**. It can be observed from Table 1 that EQC-Pro consistently outperforms the two baselines, QClique and FPCE, across all instances. Specifically, in 13 out of 20 cases, both baselines fail to produce a solution within 3 hours, whereas EQC-Pro succeeds. For example, on the soc-delicious instance, EQC-Pro completes in 0.67 seconds, while both baselines time out, implying a speedup of at least 1.6×10^4. Furthermore, when considering the better of the two baselines, EQC-Pro achieves a speedup of over 1000× in 4 instances, at least 100× in 9 instances, and over 20× in 14 instances.

Scalability Test. We conduct scalability experiments to evaluate the performance of EQC-Pro. The results demonstrate that EQC-Pro significantly outperforms the baseline algorithms. Detailed experimental settings and results are reported in Appendix D.

7.2 Comparison with Our Variants

Evaluation on Large Graphs. Table 1 reports the performance of EQC-TD and EQC-NH under $\gamma = 0.95$. We first consider the effect of different iterative frameworks by comparing EQC-Pro and EQC-TD. From Table 1, we know that EQC-Pro, benefiting from its bottom-up doubling iterative framework, can solve 13 additional instances that time out under the top-down iterative method EQC-TD. Specifically, on ca-MathSciNet, it achieves a speedup of at least 6×10^4. For the remaining 7 instances where both EQC-Pro and EQC-TD complete within the time limit, the performance of EQC-TD is comparable to that of EQC-Pro. This is mainly due to the small final solution size s^* in these cases. For instance, in soc-twitter-follows and socfb-uci-uni, the maximum size of the 0.95-EQC is only 7, resulting in a limited allowance for missing edges. This tight constraint indirectly improves the accuracy of upper bound estimations, narrowing the performance gap between EQC-Pro and EQC-TD.

We next consider EQC-Heu-Pro (Algorithm 4) by comparing EQC-Pro and EQC-NH. The integration of EQC-Heu-Pro leads to a reduction in runtime. Among the 20 instances in Table 1, 17 instances exhibit accelerated performance, with two additional time-out cases successfully solved by EQC-Pro. In most cases, incorporating EQC-Heu-Pro yields a clear efficiency gain. We now examine the few cases with less favorable outcomes. In inf-road-usa, for instance, EQC-NH outperforms EQC-Pro, consistent with our earlier observation that the heuristic component Degen-Opt, retained in EQC-NH, can already produce high-quality solutions in certain settings. The speedup is also marginal in socfb-uci-uni and socfb-konect, where EQC-Pro is faster by only about 5 seconds, despite total running times of approximately 28.15 and 22.82 seconds, respectively. This limited improvement mainly stems from the dominance of Degen-Opt in the total computation time on large-scale graphs. As both EQC-Pro and EQC-NH retain this component, the observed speedup is less obvious, despite a substantial reduction in the number of required iterations.

8 Related Work

The concept of edge-based γ-quasi-clique (γ-EQC) was defined in [2, 3], and has also been referred to as dense subgraph [27, 40], near-clique [6, 23, 38], or pseudo clique [35]. Existing exact approaches for solving MaxEQC fall into two main categories: mixed integer programming (MIP) and branch-and-bound techniques. Since MIP-based methods [29, 34, 42] are difficult to scale in practice, we focus on branch-and-bound algorithms. The first branch-and-bound algorithm for this problem was proposed by Pajouh et al. [32], who introduced a novel upper bounding technique and demonstrated strong performance. Subsequently, QClique [36] incorporated a new upper bound and showed that their approach is competitive with both MIP techniques [34, 42] and other branch-and-bound methods [32]. Although QClique achieves competitive performance, its branching scheme is identical to that of the maximal γ-EQC enumeration method PCE [41], and its pruning strategy relies on a computationally expensive upper bound. FPCE [35] also builds on PCE and specifically targets the maximal γ-EQC enumeration problem, introducing several refined bounding techniques and achieving state-of-the-art performance in both maximal γ-EQC enumeration and MaxEQC. Given that MaxEQC is NP-hard [34], various heuristics have been proposed for approximate solutions. Early approaches include GRASP-based methods and local search [3, 7]. Later studies explored greedy strategies [40] and, recently, advanced vertex selection and iterative refinement techniques [13, 26].

As mentioned earlier, there is a quasi-clique defined based on vertex degree: a degree-based γ-quasi-clique requires each vertex in the subgraph connects to at least a γ-proportion of the remaining vertices [30, 37]. DDA [33] introduced a hybrid strategy that combines MIP formulations with an iterative refinement process. More recently, Xia et al. [43] proposed IterQC, which reformulates the original problem as a series of simpler dense subgraph problems. Another closely related problem is the corresponding maximal enumeration problem, for which several branch-and-bound methods, including Quick [25] and Quick+ [22], have been developed. Most recently, Yu and Long [46] proposed the state-of-the-art method FastQC by introducing new branching and pruning strategies.

For comprehensive surveys on cohesive subgraphs, see [10, 17, 18, 21, 24].

9 Conclusion

In this paper, we proposed an efficient iterative algorithm EQC-Pro for the MaxEQC problem. Reducing MaxEQC to hereditary subproblems improves theoretical efficiency, while a bottom-up doubling framework and novel heuristics further boost performance. Experiments show that EQC-Pro outperforms state-of-the-art methods. Future work includes extending EQC-Pro to weighted graphs.

Acknowledgments

This research is partially supported by the Guangdong Basic and Applied Basic Research Foundation (No. 2025A1515060015), by the Shenzhen Science and Technology Program (No. GXWD20231129111306002 and KJZD20231023094701003), by the Major Key Project of Pengcheng Laboratory (No. PCL2024A05), and by the Key Laboratory of Interdisciplinary Research of Computation and Economics (Shanghai University of Finance and Economics), Ministry of Education.

References

[1] Source Code. https://github.com/SmartProbiotics/EQC-Pro.
[2] James Abello, Panos M. Pardalos, and Mauricio G. C. Resende. 1998. On maximum clique problems in very large graphs. In *Proceedings of the DIMACS Workshop on External Memory Algorithms*. 119–130.
[3] J. Abello, M. G. C. Resende, and S. Sudarsky. 2002. Massive quasi-clique detection. In *LATIN*. 598–612.
[4] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An $O(m)$ algorithm for cores decomposition of networks. *CoRR* cs.DS/0310049 (2003).
[5] Punam Bedi and Chhavi Sharma. 2016. Community detection in social networks. *Data Mining and Knowledge Discovery* 6, 3 (2016), 115–135.
[6] Zvika Brakerski and Boaz Patt-Shamir. 2011. Distributed discovery of large near-cliques. *Distributed Computing* 24 (2011), 79–89.
[7] Marco Brunato, Holger H. Hoos, and Roberto Battiti. 2007. On effectively finding maximal quasi-cliques in graphs. In *LION*. 41–55.
[8] Lijun Chang. 2023. Efficient maximum k-defective clique computation with improved time complexity. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 3 (2023), 1–26.
[9] Lijun Chang. 2024. Maximum defective clique computation: Improved time complexities and practical performance. *Proceedings of the VLDB Endowment* 18, 2 (2024), 200–212.
[10] Lijun Chang and Lu Qin. 2018. *Cohesive Subgraph Computation over Large Sparse Graphs*. Springer Publishing Company, Incorporated.
[11] Lijun Chang, Mouyi Xu, and Darren Strash. 2022. Efficient maximum k-plex computation over large sparse graphs. *Proceedings of the VLDB Endowment* 16, 2 (2022), 127–139.
[12] Lijun Chang and Kai Yao. 2024. Maximum k-plex computation: Theory and practice. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 1 (2024), 1–26.
[13] Jiejiang Chen, Shaowei Cai, Shiwei Pan, Yiyuan Wang, Qingwei Lin, Mengyu Zhao, and Minghao Yin. 2021. NuQClq: An effective local search algorithm for maximum quasi-clique problem. In *AAAI*. 12258–12266.
[14] Xiaoyu Chen, Yi Zhou, Jin-Kao Hao, and Mingyu Xiao. 2021. Computing maximum k-defective cliques in massive graphs. *Computers & Operations Research* 127 (2021), 105131.
[15] Qiangqiang Dai, Ronghua Li, Donghang Cui, and Guoren Wang. 2024. Theoretically and practically efficient maximum defective clique search. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 4 (2024), 1–27.
[16] Qiangqiang Dai, Rong-Hua Li, Meihao Liao, and Guoren Wang. 2023. Maximal defective clique enumeration. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 1 (2023), 1–26.
[17] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29 (2020), 353–392.
[18] Yixiang Fang, Kai Wang, Xuemin Lin, and Wenjie Zhang. 2021. Cohesive subgraph search over big heterogeneous information networks: Applications, challenges, and solutions. In *Proceedings of the ACM on Management of Data (SIGMOD)*. 2829–2838.
[19] Jian Gao, Zhenghang Xu, Ruizhi Li, and Minghao Yin. 2022. An exact algorithm with new upper bounds for the maximum k-defective clique problem in massive sparse graphs. In *AAAI*. 10174 – 10183.
[20] Shuohao Gao, Kaiqiang Yu, Shengxin Liu, and Cheng Long. 2024. Maximum k-plex search: An alternated reduction-and-bound method. *Proceedings of the VLDB Endowment* 18 (2024), 363–376.
[21] Xin Huang, Laks V. S. Lakshmanan, and Jianliang Xu. 2019. *Community Search over Big Graphs*. Morgan & Claypool Publishers.
[22] Jalal Khalil, Da Yan, Guimu Guo, and Lyuheng Yuan. 2022. Parallel mining of large maximal quasi-cliques. *The VLDB Journal* 31, 4 (2022), 649–674.
[23] Christian Komusiewicz, Manuel Sorge, and Kolja Stahl. 2015. Finding connected subgraphs of fixed minimum density: Implementation and experiments. In *SEA*. 82–93.
[24] Victor E. Lee, Ning Ruan, Ruoming Jin, and Charu Aggarwal. 2010. A Survey of Algorithms for Dense Subgraph Discovery. In *Managing and Mining Graph Data*. Springer, 303–336.
[25] Guimei Liu and Limsoon Wong. 2008. Effective pruning techniques for mining quasi-cliques. In *ECML/PKDD*. 33–49.
[26] Shuhong Liu, Jincheng Zhou, and Mingjie Lei. 2024. An optimization algorithm for maximum quasi-clique problem based on information feedback model. *PeerJ Computer Science* 10 (2024), e2173.
[27] James Long and Chris Hartman. 2010. ODES: An overlapping dense sub-graph algorithm. *Bioinformatics* 26, 21 (2010), 2788–2789.
[28] Chunyu Luo, Yi Zhou, Zhengren Wang, and Mingyu Xiao. 2024. A faster branching algorithm for the maximum k-defective clique problem. In *ECAI*. 4132–4139.
[29] Fabrizio Marinelli, Andrea Pizzuti, and Fabrizio Rossi. 2021. LP-based dual bounds for the maximum quasi-clique problem. *Discrete Applied Mathematics* 296 (2021), 118–140.
[30] Hideo Matsuda, Tatsuya Ishihara, and Akihiro Hashimoto. 1999. Classifying molecular sequences using a linkage graph with their pairwise similarities. *Theoretical Computer Science* 210, 2 (1999), 305 – 325.
[31] Tamás Nepusz, Haiyuan Yu, and Alberto Paccanaro. 2012. Detecting overlapping protein complexes in protein–protein interaction networks. *Nature Methods* 9, 5 (2012), 471–472.
[32] Foad Mahdavi Pajouh, Zhuqi Miao, and Balabhaskar Balasundaram. 2014. A branch-and-bound approach for maximum quasi-cliques. *Annals of Operations Research* 216, 1 (2014), 145–161.
[33] Grigory Pastukhov, Alexander Veremyev, Vladimir Boginski, and Oleg A. Prokopyev. 2018. On maximum degree-based γ-quasi-clique problem: Complexity and exact approaches. *Networks* 71, 2 (2018), 136–152.
[34] Jeffrey Pattillo, Alexander Veremyev, Sergiy Butenko, and Vladimir Boginski. 2013. On the maximum quasi-clique problem. *Discrete Applied Mathematics* 161, 1-2 (2013), 244–257.
[35] Ahsanur Rahman, Kalyan Roy, Ramiza Maliha, and Townim Faisal Chowdhury. 2024. A fast exact algorithm to enumerate maximal pseudo-cliques in large sparse graphs. In *SIGKDD*. 2479–2490.
[36] Celso Ribeiro and José Riveaux Merino. 2019. An exact algorithm for the maximum quasi-clique problem. *International Transactions in Operational Research* 26, 1 (2019), 129–151.
[37] Seyed-Vahid Sanei-Mehri, Apurba Das, Hooman Hashemi, and Srikanta Tirthapura. 2021. Mining Largest Maximal Quasi-Cliques. *ACM Transactions on Knowledge Discovery from Data* 15 (2021), 81:1–81:21.
[38] Shu Tadaka and Kengo Kinoshita. 2016. NCMine: Core-peripheral based functional module detection using near-clique mining. *Bioinformatics* 32, 22 (2016), 3454–3460.
[39] Brian K. Tanner, Gary Warner, Henry Stern, and Scott Olechowski. 2010. Koobface: The evolution of the social botnet. In *2010 eCrime Researchers Summit*. 1–10.
[40] Charalampos E. Tsourakakis, Francesco Bonchi, Aristides Gionis, Francesco Gullo, and Maria A. Tsiarli. 2013. Denser than the densest subgraph: Extracting optimal quasi-cliques with quality guarantees. In *SIGKDD*. 104–112.
[41] Takeaki Uno. 2010. An efficient algorithm for solving pseudo clique enumeration problem. *Algorithmica* 56, 1 (2010), 3–16.
[42] Andrey Veremyev, Oleg A. Prokopyev, Sergiy Butenko, and Eduardo L. Pasiliao. 2016. Exact MIP-based approaches for finding maximum quasi-cliques and dense subgraphs. *Computational Optimization and Applications* 64, 1 (2016), 177–214.
[43] Hongbo Xia, Kaiqiang Yu, Shengxin Liu, Cheng Long, and Xun Zhou. 2025. Maximum degree-based quasi-clique search via an iterative framework. In *SIGKDD*. 3285–3296.
[44] Haiyuan Yu, Alberto Paccanaro, Valery Trifonov, and Mark Gerstein. 2006. Predicting interactions in protein networks by completing defective cliques. *Bioinformatics* 22, 7 (2006), 823–829.
[45] Kaiqiang Yu and Cheng Long. 2021. Graph Mining Meets Fake News Detection. In *Data Science for Fake News*. 169–189.
[46] Kaiqiang Yu and Cheng Long. 2023. Fast maximal quasi-clique Enumeration: A pruning and branching co-design approach. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 3 (2023), 1–26.

A Omitted Details of Top-down Iterative Framework in Section 3

A.1 Our Top-down Iterative Baseline EQC-TD

We first detail our top-down iterative baseline algorithm EQC-TD (mentioned in Section 3) and demonstrate its correctness.

Overview. We observe that the maximum γ-EQC g^* is also a maximum k^*-defective clique in G, where $k^* = \texttt{get-k}(s^*)$, which follows directly from the definitions. Motivated by this equivalence, we design the top-down iterative framework that reduces MaxEQC to a sequence of kDC subproblems. Since the optimal size s^* is unknown, the framework starts from an initial upper bound and iteratively tightens it by solving the corresponding kDC problem. At each iteration, the obtained solution either certifies feasibility or yields a strictly smaller upper bound on s^*, thereby narrowing the search space until the optimal γ-EQC is identified. We then detail the algorithm and provide a formal proof of its correctness.

EQC-TD Algorithm. Our top-down iterative framework (named as EQC-TD) is shown in Algorithm 5. Line 1 initializes an upper bound

Algorithm 5: A Top-down Iterative Framework: EQC-TD

Input: A graph $G = (V, E)$ and a real value $0 < \gamma < 1$
Output: The size of the maximum γ-EQC in G

1 $s_0 \leftarrow$ UpperBound(G); $k_1 \leftarrow$ get-k(s_0); $i \leftarrow 1$;
2 **while** *true* **do**
3 $\quad s_i \leftarrow$ solve-defect(k_i);
4 $\quad$ **if** $k_i =$ get-k(s_i) **then**
5 $\quad\quad s^* \leftarrow s_i$; **return** s^*;
6 $\quad k_{i+1} \leftarrow$ get-k(s_i); $i \leftarrow i + 1$;

Algorithm 6: UpperBound(G)

Output: An upper bound ub of s^*

1 $ub \leftarrow (1 + \sqrt{1 + 8m/\gamma})/2$; $k \leftarrow$ get-k(ub);
2 **while** *true* **do**
3 $\quad ub \leftarrow$ ComputeUB(k);
4 $\quad$ **if** get-k(ub) $\geq k$ **then break**;
5 $\quad k \leftarrow$ get-k(ub);
6 **return** ub;

as s_0 via UpperBound (described later). This upper bound is iteratively refined within the while-loop (Lines 2-6). In each iteration, EQC-TD assumes the existence of a size-s_{i-1} γ-EQC and computes the corresponding upper bound on the number of missing edges, denoted as $k_i =$ get-k(s_{i-1}). Using this value, EQC-TD invokes solve-defect to obtain the maximum size s_i of a k_i-defective clique. We show that this procedure provides a tighter upper bound for the optimal solution s^*. The algorithm terminates when the k-sequence converges, i.e., when $k_i =$ get-k(s_i) in Line 4, and returns the corresponding size s_i as the final result in Line 5.

Correctness. We next prove the correctness of our top-down iterative framework EQC-TD for solving MaxEQC in Algorithm 5. We first establish two supporting lemmas.

Lemma 1. *Given a real number $0 < \gamma < 1$, recall that s^* denotes the size of the maximum edge-based γ-quasi-clique. If a value s satisfies the condition $s =$ solve-defect(get-k(s)), then we have $s \leq s^*$.*

Proof. Let $k =$ get-k(s) $= \left\lfloor \binom{s}{2} \times (1-\gamma) \right\rfloor$. Since we have solve-defect(k) $= s$, there exists a subgraph g of G with $|V(g)| = s$ that is a k-defective clique. Thus, we have $|E(g)| \geq \binom{s}{2} - k \geq \binom{s}{2} - \binom{s}{2}(1-\gamma) = \binom{s}{2}\gamma$, which implies that g is an edge-based γ-quasi-clique of size s. By the definition of s^* as the maximum size of any edge-based γ-quasi-clique, we conclude that $s \leq s^*$. □

Lemma 2. *Let s^* be the optimal size. Let $ub > s^*$ be an upper bound before an iteration (i.e., an execution of the while-loop in Lines 2-6 of Algorithm 1), where ub may be the s value obtained in the last iteration. Define $k =$ get-k(ub) and ub' as the s value after the iteration, i.e., $ub' =$ solve-defect(k). Then, we have $s^* \leq ub' < ub$. In other words, ub' is a strictly tighter upper bound on s^*.*

Proof. Since get-k and solve-defect are non-decreasing, by Proposition 2, we know solve-defect(get-k(s^*)) $\geq s^*$. Applying this to $ub > s^*$, we obtain $ub' =$ solve-defect(get-k(ub)) $\geq$ solve-defect(get-k(s^*)) $\geq s^*$ On the other hand, since $ub > s^*$ and with Proposition 2, we have $ub' =$ solve-defect(get-k(ub)) $< ub$. Combining the two bounds yields $s^* \leq ub' < ub$. □

Proposition 3. *Algorithm 5 can correctly computes the size of the maximum edge-based γ-quasi-clique in finite steps.*

Proof. By Lemma 1, whenever the termination condition of the while-loop (Line 4) is satisfied, the returned value satisfies $s_i \leq s^*$. By Lemma 2, as long as the termination condition is not met, the sequence $\{s_0, s_1, \ldots, s_i\}$ is strictly decreasing and remains an upper bound on s^*. Further, the s value at each iteration satisfies $s_i \geq s^*$ until the termination condition is triggered. Thus, the only possibility at termination is that $s_i = s^*$ and the algorithm can be terminated in finite steps. □

A.2 Upper Bound and Correctness

Note that the proof in the previous subsection reveals that EQC-TD essentially iteratively computes tighter upper bounds. Thus, it is possible to compute an initial upper bound before starting the iteration. In this subsection, we introduce the algorithm for solving the upper bound in EQC-TD and provide a proof of its correctness.

Upper Bound. An upper bound is commonly used in the literature for MaxEQC [34, 42], which is used to estimate the size of the maximum γ-EQC by solving $m \geq \gamma \cdot \binom{ub}{2}$, yielding $ub \leq (1 + \sqrt{1 + 8m/\gamma})/2$. However, this bound is often overly coarse. To obtain a tighter upper bound, we incorporate the coloring-based upper bounding strategy from the kDC problem [8, 15] into *an iterative framework tailored for the γ-EQC setting*. Our method for computing upper bounds for MaxEQC is based on a key observation. In our top-down iterative framework (Algorithm 5), we maintain upper bounds for the optimal values $k^* =$ get-k(s^*) and s^* at each iteration. By consistently choosing k and s as upper bounds, each iteration produces progressively tighter upper bounds. Thus, *if the k value in some iteration is an upper bound of k^*, then the upper bound computed for the corresponding* kDC *problem is also an upper bound of s^**. Based on this, we summarize our upper bound method UpperBound in Algorithm 6, where ComputeUB is an upper bound estimation procedure for kDC [8, 15]. Specifically, UpperBound first initializes a trivial ub as $(1+\sqrt{1+8m/\gamma})/2$ and sets the corresponding value of k in Line 1. Then, UpperBound iteratively invokes ComputeUB with different values of k (Lines 2-6 of Algorithm 6) until the condition get-k(ub) $\geq k$ is satisfied in Line 4. Finally, we return ub as the upper bound of s^* in Line 6.

We first present the existing ComputeUB algorithm [8, 15] for computing an upper bound for kDC, which is used in our upper bound computation algorithm UpperBound (Algorithm 6). We then prove that in our UpperBound, the iterative procedure (Lines 2-5 of Algorithm 6) produces a strictly decreasing sequence of upper bounds, each of which remains a valid upper bound of the optimal value s^*, i.e., the size of the maximum γ-EQC. This result can also be used to demonstrates the correctness of our UpperBound.

ComputeUB Algorithm. The key idea of ComputeUB is to apply a vertex coloring strategy that assigns different colors to adjacent vertices, partitioning the graph into independent sets (i.e., each color class consists non-adjacent vertices). ComputeUB focuses only on missing edges within each color class. During the selection process, vertices are greedily added such that each new vertex introduces the

Algorithm 7: ComputeUB(k) [8, 15]

Output: An upper bound of the size of the maximum k-defective clique.

1 colorCount[] $\leftarrow$ GreedyColor(G);
2 Let numColors be the total number of colors used;
3 maxCount $\leftarrow 0$;
4 **foreach** $c = 1$ to *numColors* **do**
5 maxCount $\leftarrow$ max(maxCount, colorCount[c]);
6 Perform a binary search over $l \in [0, \text{maxCount}]$ to find the largest l such that $\sum_{c=1}^{\text{numColors}} \binom{m_c}{2} \leq k$, where $m_c = \min(l, \text{colorCount}[c])$;
7 $ub \leftarrow 0$;
8 **foreach** $c = 1$ to *numColors* **do**
9 $m_c \leftarrow \min(l, \text{colorCount}[c])$;
10 $k \leftarrow k - \binom{m_c}{2}$;
11 $ub \leftarrow ub + m_c$;
12 $ub \leftarrow ub + \lfloor k/(l+1) \rfloor$;
13 **return** ub;
14 **Function** GreedyColor(G):
15 **foreach** *vertex u in reverse degeneracy order* **do**
16 Assign to u the smallest color that has not been assigned to any of its colored neighbors;
17 **return** an array containing the number of vertices assigned to each color;

fewest missing edges among already selected vertices of the same color. This process continues until the total number of missing edges reaches or exceeds the threshold k. Since missing edges between different color classes are ignored, the number of selected vertices provides an upper bound on the size of the maximum k-defective clique. We remark that since the greedy selection is based on a fixed coloring order rather than k, ComputeUB is monotonic in k: for $k' > k$, ComputeUB(k') $\geq$ ComputeUB(k).

ComputeUB is shown in Algorithm 7. The graph is first greedily colored in Line 1. Then, vertices are selected greedily to minimize new missing edges within each color class. To improve the efficiency, we use a batch-wise selection strategy: initially, up to l vertices per color class are selected, followed by at most one more vertex per class. Since the coloring order is fixed, l can be efficiently found via binary search (Lines 2-6), after which any remaining selections are made iteratively (Lines 7–12). Note that as long as the coloring order is fixed, the correctness is guaranteed. In our implementation, we use the reverse degeneracy ordering for coloring.

Correctness. We first note that ComputeUB (Algorithm 7) is monotone, i.e., for any $k_1 \geq k_2$, we have ComputeUB(k_1) $\geq$ ComputeUB(k_2). Based on this monotonicity, we present the following lemma.

Lemma 3. *During the iterative process (Lines 2-5) of Algorithm 6, the computed upper bound ub forms a non-increasing sequence and eventually terminates at a value that is an upper bound of s^*, i.e., the size of the maximum γ-EQC.*

Proof. Algorithm 6 iteratively computes a sequence of upper bounds on optimal value s^*. We prove that upon termination, the final upper bound is a valid upper bound on s^*, and that the algorithm terminates in finitely many steps. Initially, in Line 1 of Algorithm 6, the value of k is set by assuming that the optimal value is ub. Specifically, the algorithm computes the maximum number of missing edges allowed in a γ-EQC of size ub, which serves as an upper bound on the number of missing edges in the corresponding maximum k^*-defective clique, i.e., $k^* = \texttt{get-k}(s^*)$.

Consider two consecutive iterations. Let ub_1 denote the current upper bound of the optimal value s^* at the beginning of an iteration, and let k_1 be the corresponding upper bound on the number of missing edges, i.e., $k_1 \geq k^*$. Let $k_2 = \texttt{get-k}(ub_1)$ be the maximum number of missing edges associated with a γ-EQC of size ub_1, and let $ub_2 = \texttt{ComputeUB}(k_2)$ be the new upper bound computed for the next iteration. In each subsequent iteration, the algorithm updates $ub_1 \leftarrow ub_2$ and $k_1 \leftarrow k_2$. Observe that the while loop proceeds only if $k_2 = \texttt{get-k}(ub_1) < k_1$, ensuring that the value of k strictly decreases with each iteration. Since the function ComputeUB is monotonic with respect to k, $ub_2 = \texttt{ComputeUB}(k_2) \leq \texttt{ComputeUB}(k_1) = ub_1$, and thus, the sequence of upper bounds is non-increasing.

Moreover, ComputeUB returns a valid upper bound on the size of the maximum k-defective clique for the given k. By construction, $k_2 = \texttt{get-k}(ub_1) \geq \texttt{get-k}(s^*) = k^*$, and thus $ub_2 = \texttt{ComputeUB}(k_2) \geq \texttt{ComputeUB}(k^*) = s^*$ throughout the process. Since k is a non-negative integer that can only take finitely many values bounded below by k^*, and since k strictly decreases in each iteration, the algorithm must terminate after finitely many steps. Upon termination, the final ub remains a valid upper bound on s^*. □

Algorithm 8: Auxiliary Subroutines of EQC-Heu

Function AddBestVertex(S, C):
$P \leftarrow \{u \in C \mid d_G^S(u) = \max_{v \in C} d_G^S(v)\}$;
$P' \leftarrow \{u \in P \mid \text{score}(u) = \max_{v \in P} \text{score}(v)\}$;
$u \leftarrow$ a randomly chosen vertex from P';
move u from C to S;
score[u] $\leftarrow 0$;
Function RemoveWorstVertex(S, C):
$P \leftarrow \{u \in S \mid d_G^S(u) = \min_{v \in S} d_G^S(v)\}$;
$P' \leftarrow \{u \in P \mid \text{score}(u) = \min_{v \in P} \text{score}(v)\}$;
$u \leftarrow$ a randomly chosen vertex from P';
move u from S to C;
score[u] $\leftarrow 0$;
Function UpdateScore(S, C):
foreach *vertex $v \in S$* **do**
score[v] $\leftarrow$ score[v] $+ d_G^C(v) - \Delta_G$;
foreach *vertex $v \in C$* **do**
score[v] $\leftarrow$ score[v] $+ d_G^S(v)$;

B Score Functions in EQC-Heu (Algorithm 3)

For completeness, we present the auxiliary subroutines used in EQC-Heu (Algorithm 3 in Section 5). We note that our EQC-Heu uses

Algorithm 9: Our Final Algorithm: `EQC-Pro`

Input: A graph $G = (V, E)$ and a real value $0 < \gamma < 1$
Output: The size of the maximum γ-EQC in G

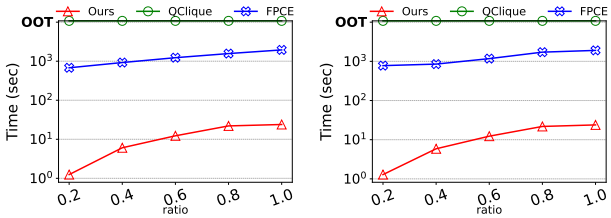
1 **global** *memo* **as** map from k to s';
2 $g \leftarrow$ `Degen-Opt`(G, γ);
3 $g \leftarrow$ `EQC-Heu-Pro`(G, g, γ);
4 $gap \leftarrow 1$; $lb, lb_{new} \leftarrow |V(g)|$; $ub \leftarrow |V|$;
5 **while** $lb + gap \leq ub$ **do**
6 $\quad s \leftarrow lb + gap$; $k \leftarrow$ `get-k`(s);
7 $\quad s' \leftarrow$ `MemoizedSearch`(k);
8 $\quad$ **if** $s' < s$ **then break**;
9 $\quad lb_{new} \leftarrow s'$;
10 $\quad gap \leftarrow 2 \times gap$;
11 $ub_{\text{new}} \leftarrow lb + gap - 1$;
12 **while** $lb_{new} \neq ub_{new}$ **do**
13 $\quad mid \leftarrow \lceil (lb_{\text{new}} + ub_{\text{new}})/2 \rceil$; $k \leftarrow$ `get-k`(mid);
14 $\quad s' \leftarrow$ `MemoizedSearch`(k);
15 $\quad$ **if** $s' \geq mid$ **then**
16 $\quad\quad lb_{\text{new}} \leftarrow mid$;
17 $\quad$ **else**
18 $\quad\quad ub_{\text{new}} \leftarrow mid - 1$;
19 **return** lb_{new};
20 **Function** `MemoizedSearch`(k):
21 $\quad$ **if** $k \in memo$ **then**
$\quad\quad$ `// memoization`
22 $\quad\quad s' \leftarrow memo[k]$;
23 $\quad$ **else**
24 $\quad\quad s' \leftarrow$ `solve-defect`(k);
25 $\quad\quad memo[k] \leftarrow s'$;
26 $\quad g \leftarrow$ the graph corresponding to the result returned from `solve-defect`(k) with size s';
27 $\quad g \leftarrow$ `EQC-Heu-Pro`(G, g, γ); `// expansion`
28 $\quad$ **return** $|V(g)|$;

the score function from [13]. The auxiliary subroutines are summarized in Algorithm 8. The procedure `AddBestVertex` (Lines 1–6) selects from the candidate set C the vertex with the largest degree towards S, breaking ties by the score function, and then moves it into S with its score reset. Symmetrically, `RemoveWorstVertex` (Lines 7–12) identifies the vertex in S with the smallest internal degree, again breaking ties by score, and moves it back to C with its score reset. Finally, `UpdateScore` (Lines 13–17) updates the scores of all vertices: vertices in S are adjusted according to their connections to C relative to Δ_G, while vertices in C are updated based on their degree towards S. Together, these subroutines support the main heuristic process by guiding vertex movements and maintaining score information during the iterations.

C Omitted Details of `EQC-Pro` in Section 6

In this section, we present the omitted details of `EQC-Pro` mentioned in Section 6. Based on two observations from Section 6, namely that (1) solutions from `solve-defect`(k) can be reused for multiple values of s and that (2) each iteration yields a valid `kDC` solution which can be used to further strengthen the lower bound for `MaxEQC`. As shown in Algorithm 9, `EQC-Pro` is an improved iterative algorithm built on the bottom-up doubling framework `EQC-BU`(Algorithm 1) and the heuristic strategy `EQC-Heu-Pro` (Algorithm 4), and further incorporates two main techniques in the `MemoizedSearch` function: *memoization* and *expansion*. First, we initiate a global memoization table (`memo`, Line 1) to cache results from previous calls to `solve-defect`(k), thus avoiding redundant computations across iterations. Subsequently, in Lines 2-3, we compute an initial heuristic solution g using `Degen-Opt` and refine it with `EQC-Heu-Pro`, which attempts to expand the solution. The size of the resulting solution is used to initialize the lower bound lb of the search space for the bottom-up iterative framework.

After initialization (Line 4), `EQC-Pro` performs the bottom-up doubling iterative search in Lines 5-19, utilizing `MemoizedSearch` (Lines 20-28) in each iteration. We reuse cached results of `solve-defect`(k) when available; otherwise, we invoke it and store the result in `memo` if its size is at least s. Moreover, `MemoizedSearch` further *expands* the corresponding solution using `EQC-Heu-Pro`, which may yield a larger feasible solution. Once the iterative search terminates, the size of the maximum γ-EQC is returned in Line 19. The correctness of `EQC-Pro` directly follows from that of Algorithm 1.



(a) $\gamma = 0.96$ (b) $\gamma = 0.98$

Figure 5: Scalability test on instance `socfb-konect`.

D Scalability Test

To evaluate scalability, we select the largest instance across the two collections and perform subgraph sampling by randomly selecting vertices with increasing probabilities and extracting the induced subgraphs. The results are presented in Figure 5. As shown, `QClique` consistently fails to solve any instance within the time limit, regardless of the sampling ratio or the value of γ, indicating its inability to handle large graphs. For `FPCE`, the runtime appears to grow nearly linearly with the number of vertices in the figure due to the use of a logarithmic scale on the y-axis. In reality, this behavior is consistent with its exponential time complexity of $O^*(2^n)$, where n is the number of vertices in the sampled subgraph.

In contrast, the runtime of `EQC-Pro` initially increases more rapidly than that of `FPCE` as the graph size grows. This is because, for smaller subgraphs, the polynomial-time components, such as `Degen-Opt` and `EQC-Heu-Pro`, dominate the overall computational cost. However, as the graph size increases and the exponential search component becomes dominant, the advantage of our improved exponential base $O^*(\beta_\kappa^n)$ becomes increasingly apparent. Consequently, for larger graphs, `EQC-Pro` grows more slowly and consistently outperforms `FPCE` across all scales.